

Basic (your first real API call) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Work through in order — don't jump ahead until you've genuinely tried.

- [Hint 1](#)
- [Hint 2](#)
- [Hint 3](#)
- [Hint 4](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 1

Simple Version

You need three things: a client (your connection to OpenAI), a list of messages (what you're saying to it), and one line to print the reply.

The client reads your API key automatically from the `OPENAI_API_KEY` environment variable, if `.env` is already loaded — you don't have to type the key into your code anywhere.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

Three pieces, matching the Core Concepts above: `client = OpenAI()` creates the client (it reads `OPENAI_API_KEY` from the environment on its own, so call `load_dotenv()` from Doc01's config pattern first if you haven't already). `client.chat.completions.create(model=..., messages=[...])` sends the request. The `messages` list needs at least one `{"role": "system", ...}` entry and one `{"role": "user", ...}` entry — the system entry sets behavior, the user entry is the actual question.

Write the three lines (client, messages list, the call) before moving to Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 2

Simple Version

The reply text is buried a few levels deep in the response object:

`response.choices[0].message.content` . There's always at least one item in `choices` , so `[0]` is safe here.

Pick two very different system prompts to compare — like "You are a formal, professional assistant" versus "You are a sarcastic pirate" — so the change in the answer is obvious.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

`response.choices` is a list because the API can technically return more than one candidate answer (controlled by an `n` parameter you're not using here) — with the default settings, there's exactly one, at index `0`. `response.choices[0].message.content` is a plain string: the model's reply text.

For the system-prompt comparison, keep the user message identical across both runs and change only the system message — that isolates the variable you're actually testing. A good pair: a neutral system prompt vs. one that asks for a very different tone or persona, so the difference is unmistakable rather than subtle.

Write the full script (client, first system prompt, call, print) before moving to Hint 3.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 3

Simple Version

```
import the OpenAI client
load the .env file

make the client

messages = [
    system message: "You are a formal, professional assistant."
    user message: "Tell me about your day."
]

send the messages to the model
print the reply text

change the system message to something very different
send again with the same user message
print the new reply text
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

def ask(system_prompt: str, user_prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content

print(ask("You are a formal, professional assistant.", "Tell me about your day."))
print(ask("You are a sarcastic pirate.", "Tell me about your day."))
```

Wrapping the call in a small `ask()` function is optional for this exercise but makes comparing two system prompts much less repetitive — try writing it this way before checking the Solution.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 4

Simple Version

Here is almost the whole thing:

```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a formal, professional assistant."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)
```

What's missing: loading `.env` first, and the second call with a different system prompt. Add those, then check the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

The same thing, with `.env` loading and a reusable function added:

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

def ask(system_prompt: str, user_prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content
```

What's missing: the two calls to `ask()` with different system prompts, and printing both. Write those yourself, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Full solution: [Show me the solution](#)